

MODULE 4 EXPERIMENT REPORT

Robot Framework Automation

Integrated experiment covering Robot Framework setup, RIDE, keyword-driven and data-driven testing, SeleniumLibrary browser automation, execution options, Jenkins scheduling, readable automation design, user-defined keywords, resource files, setup/teardown, page objects, and Gherkin-style readability.

1. Module Name and Experiment Title

Module 3.0: Robot Framework

Experiment Title: Designing, Executing, and Maintaining Browser Automation Tests Using Robot Framework and SeleniumLibrary

2. Problem Statement

Manual browser testing is repetitive and difficult to maintain when test cases are written as long procedural scripts. This experiment introduces Robot Framework as a keyword-driven automation framework and demonstrates how to create readable, reusable, maintainable, and executable browser tests using RIDE, SeleniumLibrary, command-line execution, remote browser platforms, and CI scheduling.

3. Objectives

- Install and configure Robot Framework and relevant packages.
- Create and execute simple test cases using RIDE.
- Understand keyword-driven and data-driven test case design.
- Use command-line execution, setup, teardown, tags, and run options.
- Automate browsers with SeleniumLibrary.
- Understand Robot versus Pybot terminology and execution approaches.
- Execute tests from PyCharm, command window, batch files, Task Scheduler, Sauce Labs, and Jenkins.
- Improve readability through user-defined keywords, resource files, page objects, and Gherkin-style steps.

4. Tools, Software, and Concepts Used

- Python 3.x
- Robot Framework
- RIDE
- SeleniumLibrary
- Selenium WebDriver and browser drivers
- PyCharm or VS Code

- Command Prompt / terminal
- Sauce Labs
- Jenkins
- Windows Task Scheduler
- Keyword-driven testing
- Data-driven testing
- Procedural and Gherkin-style test design
- Page Object concepts

5. Source Code and Implementation Steps

5.1 Environment Setup

```
pip install robotframework
pip install robotframework-seleniumlibrary
pip install robotframework-ride
robot --version
ride.py
```

The environment should contain Python, Robot Framework, SeleniumLibrary, a compatible browser driver, and RIDE when graphical test creation is required.

5.2 Simple Robot Framework Test Case

```
*** Settings ***
Library      SeleniumLibrary
Suite Setup   Open Browser      https://example.com      chrome
Suite Teardown  Close All Browsers

*** Test Cases ***
Verify Example Page
    Maximize Browser Window
    Title Should Be      Example Domain
    [Teardown]          Capture Page Screenshot
```

Robot Framework uses a tabular syntax. Settings define libraries and lifecycle actions, while test cases contain keywords and arguments.

5.3 Keyword-Driven and Data-Driven Testing

```
*** Keywords ***
Login With Valid Credentials
    [Arguments]      ${username}      ${password}
    Input Text        id=username      ${username}
    Input Password    id=password      ${password}
    Click Button      id=login

*** Test Cases ***
Login Test
    Login With Valid Credentials      testuser      secret
```

Keyword-driven testing expresses actions as reusable keywords. Data-driven testing runs the same workflow with different input values, which can be stored in variables, tables, external files, or resource files.

5.4 Command Line, Setup and Teardown

```
robot --outputdir results tests/  
robot -t "Verify Example Page" tests/  
robot --include smoke tests/  
robot --exclude regression tests/
```

Suite setup runs before the suite, test setup runs before each test, test teardown runs after each test, and suite teardown runs after the suite. These mechanisms support predictable resource preparation and cleanup.

5.5 Browser Automation Using SeleniumLibrary

```
*** Settings ***  
Library      SeleniumLibrary  
  
*** Test Cases ***  
Browser Interaction  
    Open Browser      https://example.com      chrome  
    Maximize Browser Window  
    Page Should Contain      Example Domain  
    Capture Page Screenshot  
    Close All Browsers
```

SeleniumLibrary provides keywords for opening browsers, locating elements, entering text, clicking controls, validating page content, taking screenshots, and closing browsers.

5.6 Script Running Options

Robot and Pybot refer to Robot Framework execution commands in different versions or historical usage contexts. Current environments generally use the robot command.

- Run from PyCharm using a configured run configuration.
- Run from Command Prompt or terminal.
- Create a batch file containing the robot command.
- Use Windows Task Scheduler for scheduled execution.
- Run one test case, multiple suites, or tests selected by tags.
- Use output directories, variables, log levels, and include/exclude options.

5.7 Sauce Labs Browser Execution

Remote browser execution can be configured using Selenium Remote WebDriver and Sauce Labs capabilities. The test selects browser, operating system, and browser version, then sends commands to the remote execution environment. Credentials and access keys should be stored securely rather than hard-coded.

5.8 Jenkins Integration and Scheduling

A Jenkins job can execute Robot Framework tests as part of a CI/CD pipeline. The job may run manually, on a schedule, after source-control changes, or as part of another pipeline. Test results, logs, screenshots, and Robot Framework reports should be archived after each build.

```
robot --outputdir results tests/  
# Archive results, log.html, report.html and output.xml in Jenkins
```

5.9 Improving Readability

Procedural scripts describe detailed actions in sequence. Gherkin-style descriptions use business-readable language such as Given, When, and Then. Robot Framework can support this readability through meaningful user-defined keywords.

```
*** Keywords ***
User Is On Login Page
    Go To      ${LOGIN_URL}

User Logs In With Valid Credentials
    Input Text    id=username    ${USERNAME}
    Input Password id=password    ${PASSWORD}
    Click Button  id=login

Dashboard Should Be Visible
    Page Should Contain    Dashboard
```

5.10 Resource Files and Page Objects

Resource files store shared keywords, variables, and locators so that multiple suites can reuse them. Page Object concepts separate page-specific locators and interactions from the test case layer, reducing duplication and simplifying maintenance when the UI changes.

```
*** Settings ***
Resource    resources/login_keywords.robot
Resource    resources/common_keywords.robot
```

6. Output with Clearly Labelled Screenshots

The following screenshots are illustrative documentation samples showing expected framework views and execution stages. They are not verified live execution captures from a specific environment.

Figure 1: Robot Framework Environment Setup

Figure 1: Robot Framework Environment Setup

Python and Robot Framework installed successfully
Packages: robotframework, robotframework-seleniumlibrary
RIDE installed for graphical test development
Browser driver and environment variables configured

Observation: Illustrative expected output for this stage.

Figure 2: RIDE Simple Test Case

Figure 2: RIDE Simple Test Case

Test Case: Open Example Website
Keywords: Open Browser, Maximize Browser Window, Close All Browsers
Test case organized in a readable tabular format

Observation: Illustrative expected output for this stage.

Figure 3: Keyword and Data-Driven Test

Figure 3: Keyword and Data-Driven Test

Reusable user-defined keyword: Login With Valid Credentials
Test data supplied through variables or data tables
Multiple input combinations executed using the same keyword

Observation: Illustrative expected output for this stage.

Figure 4: Command Line and Setup/Teardown

Figure 4: Command Line and Setup/Teardown

robot --outputdir results tests/
Suite Setup: Open Application
Test Teardown: Capture Page Screenshot
Suite Teardown: Close All Browsers

Observation: Illustrative expected output for this stage.

Figure 5: Browser Automation Using SeleniumLibrary

Figure 5: Browser Automation Using SeleniumLibrary

```
Open Browser    https://example.com    chrome
Input Text     id=username    testuser
Click Button    id=login
Page content and expected title validated
```

Observation: Illustrative expected output for this stage.

Figure 6: Script Running Options

Figure 6: Script Running Options

```
Robot command used from terminal or PyCharm
Single test: robot -t 'Test Name' tests/
Multiple suites: robot suite1.robot suite2.robot
Options include tags, variables, output directory and log level
```

Observation: Illustrative expected output for this stage.

Figure 7: Execution on Sauce Labs Browsers

Figure 7: Execution on Sauce Labs Browsers

Remote browser execution configured using Selenium Remote WebDriver
Browser, operating system and version selected as capabilities
Execution status and test results collected from remote run

Observation: Illustrative expected output for this stage.

Figure 8: Jenkins Scheduling and Execution

Figure 8: Jenkins Scheduling and Execution

Jenkins job configured with Robot Framework command
Build trigger: schedule / SCM / manual
Reports archived after execution
Failed builds marked using test result status

Observation: Illustrative expected output for this stage.

Figure 9: Readability with User-Defined Keywords

Figure 9: Readability with User-Defined Keywords

Procedural steps refactored into meaningful business keywords
Keywords moved to resource files for reuse
Setup and teardown maintained separately from test logic

Observation: Illustrative expected output for this stage.

Figure 10: Gherkin Style and Page Object Overview

Figure 10: Gherkin Style and Page Object Overview

Given the user is on the login page
When the user enters valid credentials
Then the dashboard should be displayed
Page Object concepts separate locators and actions from test cases

Observation: Illustrative expected output for this stage.

7. Result, Observation and Conclusion

The experiment explains how Robot Framework can be used to create readable and reusable browser automation. RIDE supports graphical test authoring, SeleniumLibrary

provides browser interaction keywords, and setup/teardown mechanisms manage the test lifecycle. Command-line options enable selective and advanced execution.

User-defined keywords, resource files, page object concepts, and Gherkin-style language improve maintainability. Integration with Sauce Labs and Jenkins supports remote browser execution, continuous integration, and scheduled automation.